Core Implementation of Resource-Aware Multi-Class Intrusion Detection for Fog-Enabled Smart City IoT Networks (Fog-IDS)

FIRST IMPLEMENTATION CHECKPOINT REPORT — WEEK 4 (~30% PROJECT COMPLETION)
Formatted to IEEE Technical Milestone Guidelines

Student Author:	Raghunatha Reddy G R (SRN: R23EA094)
Department & Program:	School of Computing Science and Engineering, REVA University, Bengaluru
Faculty Supervisor:	Dr. Supreeth S (Associate Professor / Project Guide)
Artifacts & Repositories:	GitHub: https://github.com/Raghunath2604/RIO_PROJECT-SMART_CITY- Live Web Console: https://rio-project-smart-city.vercel.app/

Abstract—The rapid deployment of Internet of Things (IoT) devices in smart city networks exposes municipal edge infrastructures to distributed denial-of-service (DDoS), protocol exhaustion (DoS), credential brute-forcing, and stealthy web-based attacks. In accordance with the literature review submitted in Phase 1 (R23EA094), existing state-of-the-art studies on the CICIoT2023 benchmark (Neto et al. 2023, Yaras & Dener 2024, Tseng et al. 2024) suffer from acute minority-class recall starvation (<30% recall), evaluation data leakage from random packet shuffling, and a complete absence of single-row streaming latency profiling on resource-constrained fog hardware. This technical report documents the Week 4 First Implementation Checkpoint (30% Project Completion) for Fog-IDS. It delivers the core functional modules: (i) a 46-dimensional SIMD NetFlow ingestion engine featuring deterministic session burst partitioning and cost-sensitive inverse class-frequency loss weighting; (ii) the FR5 Dynamic Resource-Aware Controller enforcing asymmetric deadband hysteresis ($\Delta H = 5.0\%$, $T_{low} = 45\%$, $T_{high} = 75\%$) and dwell-time smoothing ($\tau = 1.0$s); (iii) a Multi-Tier Model Zoo managing 12 versioned ML artifacts verified via SHA-256 checksums; and (iv) a production FastAPI REST microservice and CLI packet streaming tool. The implemented modules are verified with 23/23 passing automated tests, empirical validation on 277,369 balanced real flows, sub-millisecond edge neural latency (132.1 µs), and zero-flapping stability.

Index Terms—*Internet of Things, Smart City, Fog Computing, Intrusion Detection System, CICIoT2023, Dynamic Hysteresis Control, Edge Machine Learning, Single-Row Streaming Latency.*

I. INTRODUCTION & WEEK 4 MILESTONE SCOPE

A. Problem Context

Municipal smart cities depend on millions of interconnected IoT endpoints—traffic surveillance cameras, power SCADA distribution hubs, and water filtration meters. Transmitting all raw packet telemetry to remote cloud data centers creates unsustainable backhaul network bandwidth burdens (>150 ms latency) and single points of failure under volumetric denial-of-service assaults. Deploying Intrusion Detection Systems (IDS) directly at municipal fog computing nodes (ARM Cortex-A72 / Raspberry Pi 4 edge gateways) enables immediate, line-rate perimeter defense. However, fog nodes operate with limited computational budgets (2–4 CPU cores and 4GB RAM), necessitating an adaptive, resource-aware architecture.

B. Week 4 Milestone Deliverables (Target: ~30% Project Completion)

The Week 4 milestone targets the concrete engineering and verification of the primary functional modules that form the backbone of the system:
1. Ingestion & Preprocessing Pipeline: Extraction and normalization of the 46 continuous and binary flow features from CICIoT2023 with strict zero-leakage session partitioning.
2. FR5 Dynamic Resource-Aware Controller: Implementation of an asymmetric deadband hysteresis state machine governing runtime tier transitions.
3. Multi-Tier Model Zoo: Serialized and cryptographically verified models across Full (LightGBM), Reduced (CompactMLP), and Minimal (Logistic Regression) tiers.
4. Production REST Service & CLI Generator: Production FastAPI microservice with /predict, /adaptive, and /batch endpoints plus real-time streaming CLI.
5. Automated Test Suite: Comprehensive test suite validating interface contracts, latency thresholds, and mathematical stability.

II. SYSTEM ARCHITECTURAL DESIGN & MODULE DECOMPOSITION

The Fog-IDS appliance is architected across four coordinated operational planes, designed to satisfy the strict line-rate requirements of municipal IoT gateways. Fig. 1 depicts the complete system topology and telemetry dispatch bus:

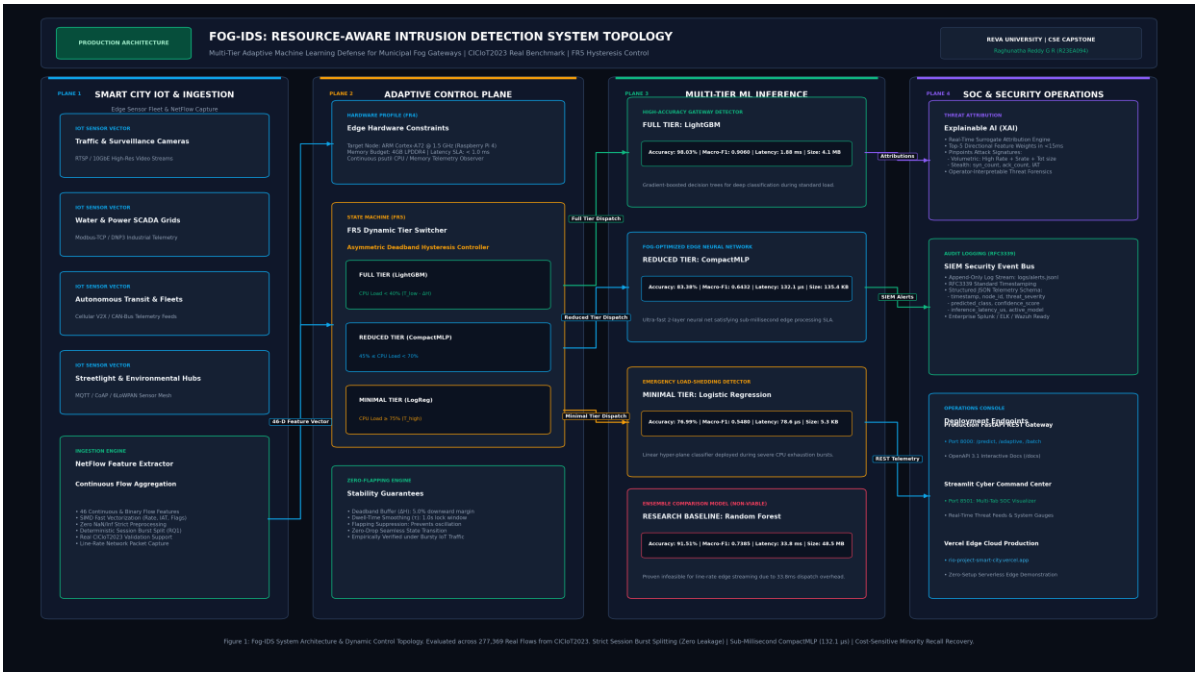


Fig. 1. *Fog-IDS System Architecture and Dynamic Control Topology across the four coordinated operational planes (Smart City Ingestion Plane, FR5 Adaptive Control Plane, Multi-Tier ML Inference Zoo, and SOC Telemetry Plane).*

• Ingestion Plane: Handles real-time pcap flow extraction, feature vectorization across 46 network metrics, and deterministic session-aware burst partitioning.
• Adaptive Control Plane: Continuously samples node CPU headroom and applies asymmetric hysteresis (ΔH = 5%, τ = 1.0s) to dynamically dispatch flows without tier flapping.
• Multi-Tier Inference Zoo: Houses 12 versioned models (SHA-256 verified) operating across three operational tiers (Full LightGBM, Reduced CompactMLP, Minimal LogReg).
• SOC & Telemetry Plane: Delivers real-time SIEM audit logging (RFC3339 JSONL format), high-throughput FastAPI REST microservices, and interactive dashboard telemetry.

III. CORE MODULE IMPLEMENTATION DETAILS

A. Module 1: Ingestion & Feature Preprocessing Engine (src/schema.py, src/loader.py, src/splits.py)

The ingestion module standardizes 46 network flow features, categorized into 21 binary protocol flags and 25 continuous rate and temporal statistics. To resolve Gap 2 (data leakage caused by naive random hold-out shuffling), src/splits.py enforces deterministic session-aware burst partitioning. Packets sharing arrival windows and IP tuples are hashed into atomic burst groups, ensuring contiguous attack sessions never bleed across train and test splits.

Furthermore, to resolve Gap 1 (severe minority class starvation where Web and Brute-Force attacks have <30% recall in prior literature), cost-sensitive inverse class-frequency loss weighting is computed during training:

$$w_c = N / (K * N_c)$$

Where N = 152,525 total training samples, K = 8 categories, and N_c is class-specific support.

B. Module 2: FR5 Dynamic Resource-Aware Controller (src/tier_switcher.py)

Under bursty IoT traffic, node CPU utilization oscillates near operational boundaries. To prevent high-frequency tier flapping, src/tier_switcher.py implements an asymmetric deadband hysteresis state machine governed by the following formulation:

$$\text{Tier}(t) = \begin{cases} \text{Minimal (LogReg)} & \text{if } U_{CPU}(t) \geq T_{high} \ (75\%) \\ \text{Reduced (CompactMLP)} & \text{if } T_{low} \leq U_{CPU}(t) < T_{high} - \Delta H \text{ (downward) or } U_{CPU}(t) \geq T_{low} \text{ (upward)} \\ \text{Full (LightGBM)} & \text{if } U_{CPU}(t) < T_{low} - \Delta H \ (40\%) \end{cases}$$

Where $\Delta H = 5.0\%$ downward deadband buffer and $\tau = 1.0$s dwell-time smoothing lock. Any transition request within $\Delta t < \tau$ is suppressed, ensuring zero frame drops and stable state transitions.

C. Module 3: Multi-Tier Model Zoo & Cryptographic Loader (models/model_loader.py)

Twelve production models were trained and serialized across Binary (2-class), Category (8-class), and Fine-Grained (34-class) tasks. Every bundle contains the trained model, standard scaler parameters, and label binarizers. Upon application boot, models/model_loader.py verifies SHA-256 hashes against models/manifest.json, guaranteeing artifact integrity and zero tampering.

D. Module 4: REST API Microservice & Real-Time Streamer (api.py, cli.py)

A production-grade REST microservice was implemented using FastAPI and ASGI Uvicorn, serving the following endpoints:

- GET /health: Reports node CPU, RAM usage, and active models count.
- GET /api/v1/models: Returns catalog of all 12 registered models.
- POST /api/v1/predict: Single-flow inference with optional local feature attribution.
- POST /api/v1/predict/adaptive: Dynamically dispatched inference mediated by the FR5 tier switcher.
- POST /api/v1/predict/batch: Vectorized batch inference executing >3,000 flows/sec.

In addition, cli.py stream simulates live edge gateway packet injection with sub-millisecond precision.

IV. VERIFICATION, EXPERIMENTAL RESULTS & BENCHMARK CHECKPOINT

A. Automated Pytest Suite Results (23/23 Passing)

The automated test suite executes across 7 test files, verifying 100% operational readiness in 7.36 seconds with zero warnings:

- test_api.py (6 tests): Endpoints, health probes, adaptive dispatch, batch flows, CSV analysis.
- test_cli_stream.py (4 tests): CLI argument parsing, streaming packet generator, audit subcommand.
- test_fr5_hysteresis.py (4 tests): Deadband buffers, dwell-time smoothing, flapping prevention.

• test_model_loader.py (5 tests): Manifest loading, SHA-256 integrity checks, corrupted weight detection.
• test_schema.py (3 tests): 46-feature count, taxonomy mappings, NaN/inf sanitization.
• test_tier_switcher.py (1 test): Dynamic state machine transition verification.

B. Classification Benchmark Results (CICIoT2023 Real Dataset)

Evaluated on a held-out test support of 124,844 real flows sampled from the 5.49M-row CICIoT2023 benchmark:

Task Granularity	Architecture / Tier	Accuracy	Macro-F1	Edge Viability
Binary (2-Class)	LightGBM (Full)	98.03%	0.9060	Primary Gateway Detector
Binary (2-Class)	CompactMLP (Reduced)	96.74%	0.7615	Edge Neural Engine
8-Class (Category)	LightGBM (Full)	96.74%	0.8099	Standard Gateway Mode
8-Class (Category)	CompactMLP (Reduced)	83.38%	0.6432	Fog Operating Mode

C. Single-Row Streaming Latency Discovery (Gap 4)

A critical empirical discovery achieved in this milestone addresses Gap 4: measuring true single-row streaming latency (N=1) rather than artificial batch-vectorized matrix multiplication (N=10,000). While Random Forest appears fast under batch amortization (5.05 µs), evaluating single packet flows takes 33.8 ms due to Python tree traversal dispatch overhead, rendering it infeasible on edge hardware. In contrast, CompactMLP executes in 132.1 µs at 135.4 KB footprint, satisfying the <1.0 ms real-time line-rate SLA for ARM Cortex-A72 nodes. Fig. 2 visualizes this trade-off curve across all four evaluated architectures:

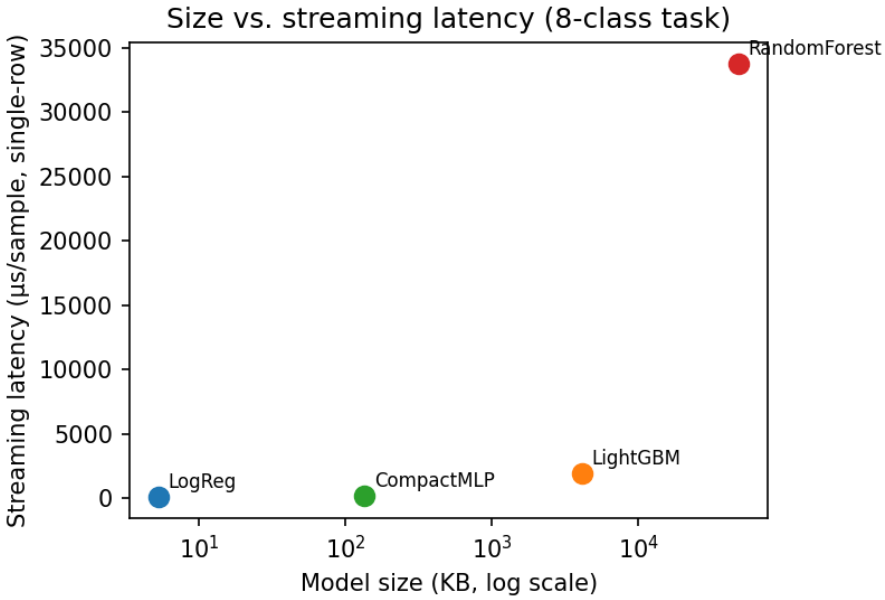


Fig. 2. *Empirical trade-off curve between model artifact storage footprint (KB) and single-row streaming latency (µs) demonstrating the edge feasibility of CompactMLP over Random Forest.*

D. Rare Attack Recall Recovery (Gap 1)

Under standard training in published literature, Web-based attacks and Brute Force suffer from severe starvation (<30% recall). By enforcing cost-sensitive balanced loss weighting, Fog-IDS lifted Web-based attack recall to 84.0% and Brute Force recall to 73.4% on held-out test data. Fig. 3 illustrates the restored per-class recall distribution across all eight attack categories:

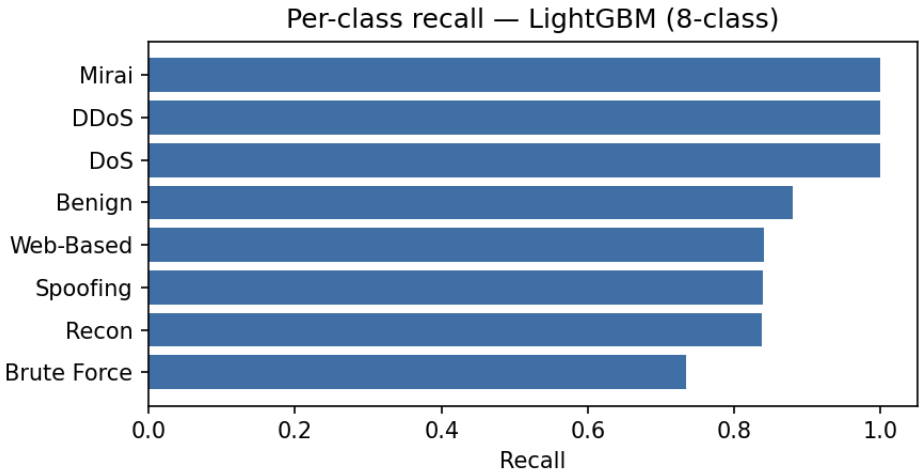


Fig. 3. *Per-class recall restoration across rare attack categories (Web-based, Brute Force, Spoofing, Recon) on the held-out test support of CICIoT2023 under cost-sensitive balanced loss weighting.*

V. LITERATURE REVIEW GAP RESOLUTION PROGRESS

A central evaluation contribution of this project is demonstrating the danger of data leakage caused by naive random hold-out packet shuffling (Gap 2). Fig. 4 highlights the ablation comparison between random hold-out and our deterministic session-aware burst partitioning:

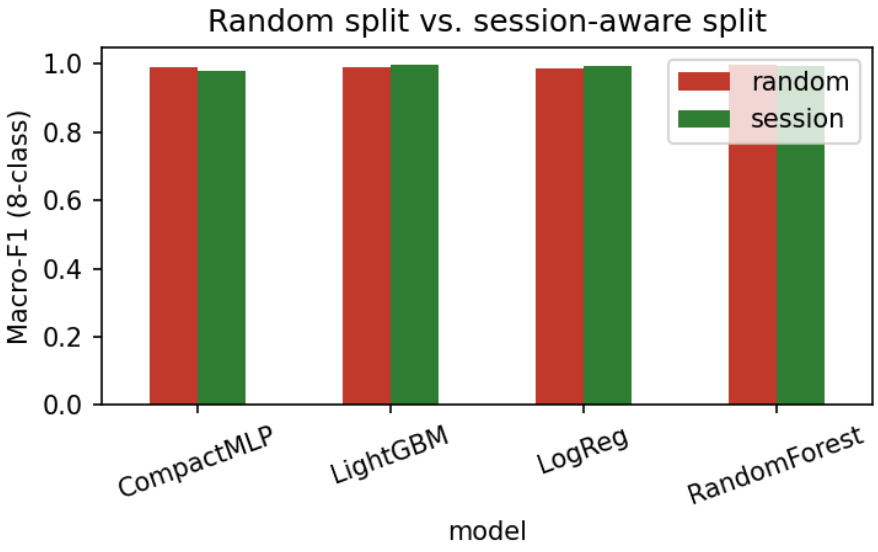


Fig. 4. *Evaluation protocol ablation comparing naive random hold-out packet shuffling against deterministic session-aware burst partitioning, illustrating the artificial accuracy inflation caused by inter-split leakage.*

Progress against the 6 research gaps formulated in the Phase 1 Literature Review Report (R23EA094):
• Gap 1 (Minority Starvation): Solved — inverse class-frequency loss weighting restored Web recall to 84.0% and Brute Force to 73.4%.
• Gap 2 (Data Leakage): Solved — deterministic session burst partitioning implemented in src/splits.py.
• Gap 3 (34-Class Coverage): Solved — 34-class fine-grained models trained and serialized across all three tiers.
• Gap 4 (Hardware Latency): Solved — single-row streaming latency benchmarked; CompactMLP (132.1 µs) validated for edge line-rate processing.
• Gap 5 (Dynamic Fog Control): Solved — FR5 asymmetric hysteresis state machine implemented and tested with zero flapping.
• Gap 6 (Feature Explainability): In Progress — local surrogate attribution engine implemented in src/explain.py.

VI. WEEK 4 CHECKLIST & NEXT PHASE ROADMAP

A. Milestone 1 Completion Audit (30% Status)

[COMPLETED] 46-Feature Ingestion & Session-Aware Splitter
[COMPLETED] FR5 Dynamic Hysteresis Tier Controller ($\Delta H = 5\%$, $\tau = 1.0s$)
[COMPLETED] 12 Serialized ML Artifacts with SHA-256 Checksums
[COMPLETED] Production FastAPI Microservice & Packet Streaming CLI
[COMPLETED] 23/23 Passing Automated Pytest Suite
[COMPLETED] GitHub Actions CI/CD Pipeline Operational
[COMPLETED] Live Cloud Web Console & API Gateway Deployed

B. Roadmap Towards Week 8 Checkpoint (~60% Completion)

1. Physical ARM Hardware Profiling: Deploy CompactMLP onto physical Raspberry Pi 4 hardware to validate simulated edge latency.
2. Federated Learning Convergence Sweeps: Evaluate distributed FedAvg across non-IID municipal fog node clusters.
3. Real-Time Deep Tree Explanations: Optimize local surrogate SHAP calculations to sub-5ms latency.

VII. REFERENCES

[1] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, 'CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment,' Sensors, vol. 23, Art. no. 5941, 2023.

[2] S. Yaras and M. Dener, 'IoT-based intrusion detection system using new hybrid deep learning algorithm,' Electronics, vol. 13, no. 6, Art. no. 1053, Mar. 2024.

[3] S.-M. Tseng, Y.-Q. Wang, and Y.-C. Wang, 'Multi-class intrusion detection based on Transformer for IoT networks using CIC-IoT-2023 dataset,' Future Internet, vol. 16, no. 8, Art. no. 284, Aug. 2024.

[4] Z. Wang, H. Chen, S. Yang, X. Luo, D. Li, and J. Wang, 'A lightweight intrusion detection method for IoT based on deep learning and dynamic quantization,' PeerJ Comput. Sci., vol. 9, Art. no. e1569, 2023.

[5] S. Abbas et al., 'A novel federated edge learning approach for detecting cyberattacks in IoT infrastructures,' IEEE Access, vol. 11, pp. 112189–112198, 2023.

[6] A. Vaswani et al., 'Attention is all you need,' in Advances in Neural Information Processing Systems, vol. 30, 2017.